ENHANCED DATA MODELS AND ADVANCED APPLICATIONS

Active Database Rules and Triggers

Outline of the chapter

0. Motivation
1. Active database rules
2. The ECA model
3. Semantic issues in trigger handling
4. Triggers in SQL:1999
5. Row-level after triggers
6. Row-level before triggers
7. Statement-level triggers
8. Disabling triggers
9. Potential applications for triggers

0. Motivation

- Triggers arise naturally in applications that require **active databases**.
- Active databases are databases that must react to various *external events*.
- In these applications, the type of possibly external events are known but their exact timings are not.
- These features make triggers a good paradigm for these applications.
- Triggers are now part of the SQL:1999 standard.

1. Active Database Rules

- **Active database rules** are rules that are activated by events that occur and can initiate certain actions that have been specified in the rule declaration if certain conditions are met.

- **Active databases** are data bases that provide additional functionality for specifying active rules.

2. The ECA Model (1)

- A model for specifying active database rules is referred to as **Event-Condition-Action (ECA)** model.

- A rule in the ECA model has three parts:

1. The **event** (or events) that activates the rule.
   An event is a *change to the DB* (an insert, delete, or update statement). In the general model, it can also be a *temporal event*, or another kind of *external event*.

2. The **condition** that determines whether the rule action should be executed. The condition is *optional*.
   If *no condition is specified*, the action will be executed once the triggering event occurs.
   If *a condition is specified*, it is first evaluated, and only if it evaluates to true will the rule action be executed.

2. The ECA Model (2)

3. The **action** to be taken. It is usually a *sequence of SQL statements*, or *a host language procedure* that will be automatically executed.

   The action can examine the answer of the condition part of the rule, refer to old and new values of tuples modified by the statement (the event part) triggering the rule, make changes to the DB, execute DDL commands (e.g create tables), and transaction-oriented commands (e.g. commit) etc.

Therefore, an active rule has the structure:

ON *event* IF *condition* THEN *action*

- The concept of a **trigger** is a technique for specifying certain types of active database rules.

3.1. Trigger execution

- An important issue is *when the action part of a rule executes in relation to the statement that triggers the rule*.

  Depending on what the rule does, we may want its action to execute *before* changes are made to the DB or *afterwards*.

3.2. Trigger granularity (1)

- The issue here is what constitutes an event.

- **Row level granularity** assumes that *a change to a single row is an event*. The trigger is considered *for every row affected by a data change operation*.

- **Statement level granularity** assumes that *events are statements* (e.g. INSERT, DELETE, UPDATE) *not* the individual tuple-level changes they make.

  For instance, an update statement that makes not changes (because the condition in the WHERE clause affects not tuples) is an event that can cause a statement level trigger to execute.

3.2. Trigger granularity (2)

- At row level granularity, a trigger might need to know the *old* and the *new values* of the *affected tuples* so it can test the condition properly.

- At statement level granularity, updates are collected in temporary structures, such as OLD TABLE and NEW TABLE. This allows the trigger to query both tables and act on the basis of the results.

3.3. Triggers and integrity constraints

- We have discussed how to specify referential triggered actions (such as ON DELETE CASCADE) in SQL on referential integrity constraints that the system should take to restore integrity.

- *These actions can be viewed as triggers* with very strict semantics.

- At the end of the execution the integrity of the database must be restored.

- The situation is complicated by the fact that a compensating action might activate other triggers that can cause violations of referential integrity.
  The scheduling of all of these triggers must have the goal of ultimately restoring the integrity constraint.

4. Triggers in SQL:1999 (1)

- *Triggering events*. an event can be the execution of an SQL INSERT, DELETE, an UPDATE statement as a whole or a change to individual tuples made by such a statement.

- *Trigger condition*. Any condition allowed in the WHERE clause of an SQL statement.

- *Trigger action*. An SQL query, a DELETE, INSERT, UPDATE, or ROLLBACK statement, or a program written in the language of SQL's *persistent stored modules* (*SQL/PSM*) which integrates procedural control statements with SQL query and update statements.

4. Triggers in SQL:1999 (2)

- *Trigger execution*. Execution can be specified to be *before* or *after* the triggering event.

- *Trigger granularity*. Row-level and statement-level granularities are both available.

5. Row-level after triggers (1)

- We illustrate row-level triggers with specific examples.

  Oracle triggers are close to the way triggers are specified in the SQL:1999 standard.

- Consider a simplified variation of the COMPANY example DB:

EMPLOYEE

NAME	SSN	SALARY	DNO	SUPERVISOR_SSN

DEPARTMENT

DNAME	DNO	TOTAL_SAL	MANAGER_SSN

- DNO in EMPLOYEE is a foreign key to DEPARTMENT.
- SUPERVISOR_SSN is a (recursive) foreign key to EMPLOYEE.
- DNO in EMPLOYEE can have a NULL value (indicating that an employee can be temporarily unassigned to a department).

5. Row-level after triggers (2)

- Attribute TOTAL_SAL in DEPARTMENT is a <u>derived attribute</u>: *its value is the sum of salaries of all the employees assigned to a particular department.*

- Maintaining a correct value of TOTAL_SAL can be done via a trigger.

  We first have to determine the **events** that cause a change in the value of TOTAL_SAL, which are:

  1. Inserting one or more new employee tuples.
  2. Changing the salary of one or more existing employees.
  3. Changing the assignment of existing employees from one department to another.
  4. Deleting one or more employee tuples.

5. Row-level after triggers (3)

- Case 1.

  Event: Inserting one or more new employee tuples.

  We need to recompute TOTAL_SAL only if the value for DNO of the new employee(s) is not NULL. This is the **condition** to be checked.

  The **action** is to automatically update the value of TOTAL_SAL for the employee's department to reflect the newly inserted employee's salary.

5. Row-level after triggers (4)

- Trigger **R1**.

```
CREATE TRIGGER TOTALSAL1

AFTER INSERT ON EMPLOYEE
FOR EACH ROW

WHEN (NEW.DNO IS NOT NULL)

UPDATE DEPARTMENT
SET TOTAL_SAL = TOTAL_SAL +  NEW.SALARY
WHERE DNO = NEW.DNO;
```

- The **CREATE TRIGGER** statement specifies a *trigger name* for R1
- The **AFTER** clause specifies that the trigger will be triggered *after* the events that trigger the rule occur.
- The *triggering events* -- an insert of a new employee -- are specified **following the AFTER** keyword.
- The **ON**-clause specifies the *relation on which the trigger is specified* -- relation EMPLOYEE.

5. Row-level after triggers (5)

- Trigger **R1**.

```
CREATE TRIGGER TOTALSAL1

AFTER INSERT ON EMPLOYEE
FOR EACH ROW

WHEN (NEW.DNO IS NOT NULL)

UPDATE DEPARTMENT
SET TOTAL_SAL = TOTAL_SAL +  NEW.SALARY
WHERE DNO = NEW.DNO;
```

- The *optional* keyword **FOR EACH ROW** specifies that the trigger will be triggered *once for each row that is affected by the triggering event*.
- The optional **WHEN**–clause specifies a *condition* that needs to be checked after the rule is triggered but before the action is executed.
- Finally, the **action** to be taken typically contains one or more SQL statements or calls to execute external procedures.

5. Row-level after triggers (6)

- The basic events that can be specified for triggering the triggers are the standard SQL data change commands INSERT, DELETE and UPDATE.
  They are specified by the keywords **INSERT**, **DELETE**, and **UPDATE**.

- In the case of UPDATE one can specify the attributes to be updated by using the keyword **OF**
  -- for instance, **UPDATE OF** SALARY, DNO.

- The keywords **NEW** and **OLD** are used to *refer to the tuples that have been inserted, deleted, or updated by the triggering event*.

  NEW refers to a newly inserted, or newly updated tuple.

  OLD refers to a deleted tuple or to a tuple before it was updated.

5. Row-level after triggers (7)

- Case 2.

  Event: Changing the salary of one or more existing employees.

  We need to recompute TOTAL_SAL only if the new (or old) value for DNO of the updated employee tuple is not NULL. This the **condition** to be checked.

  The **action** is to automatically update the value of TOTAL_SAL for the employee's department to reflect the newly updated employee's salary.

5. Row-level after triggers (8)

- Trigger **R2**.

```
CREATE TRIGGER TOTALSAL2

AFTER UPDATE OF SALARY ON EMPLOYEE
FOR EACH ROW

WHEN (NEW.DNO IS NOT NULL)

UPDATE DEPARTMENT
SET TOTAL_SAL = TOTAL_SAL +  NEW.SALARY - OLD.SALARY
WHERE DNO = NEW.DNO;
```

- Notice the use of **UPDATE OF**

- Notice the use of the keywords **NEW** and **OLD** to refer to the newly updated tuple and to the tuple before it was updated, respectively.

5. Row-level after triggers (9)

- Case 3.

  Event: Changing the assignment of existing employees from one department to another.

  No **condition** is needed. We will always execute the action to maintain the value of TOTAL_SAL correctly.

  A twofold **action** is needed; one to update the TOTAL_SAL of the employee's old department and the other to update the TOTAL_SAL of the employee's new department.

5. Row-level after triggers (10)

- Trigger **R3**.

```
CREATE TRIGGER TOTALSAL3

AFTER UPDATE OF DNO ON EMPLOYEE
FOR EACH ROW

BEGIN ATOMIC
    UPDATE DEPARTMENT
    SET TOTAL_SAL = TOTAL_SAL +  NEW.SALARY
    WHERE DNO = NEW.DNO;
    UPDATE DEPARTMENT
    SET TOTAL_SAL = TOTAL_SAL -  OLD.SALARY
    WHERE DNO = OLD.DNO;
END;
```

- Notice the absence of the WHEN-clause.

5. Row-level after triggers (11)

- Case 4.

  Event: Deleting one or more employee tuples.

  We need to recompute TOTAL_SAL only if the value for DNO of the deleted employee tuple is not NULL. This is the **condition** to be checked.

  The **action** is to automatically update the value of TOTAL_SAL for the employee's department to reflect the deleted employee's salary.

5. Row-level after triggers (12)

- Trigger **R4**.

```
CREATE TRIGGER TOTALSAL4

AFTER DELETE ON EMPLOYEE
FOR EACH ROW

WHEN (OLD.DNO IS NOT NULL)

UPDATE DEPARTMENT
SET TOTAL_SAL = TOTAL_SAL -  OLD.SALARY
WHERE DNO = OLD.DNO ;
```

6. Row-level before triggers (1)

- **AFTER triggers**. AFTER triggers execute entirely after the triggering event has applied its changes to the database.
  They are allowed to make changes to the database and thus can activate other triggers (which can cause a chain reaction).

- **BEFORE triggers**. All BEFORE triggers execute entirely before the triggering events.

- One type of BEFORE triggers are not allowed to modify the database, but can only test the condition specified in the WHEN clause and either *accept* or *abort* the triggering transaction.

  Since these BEFORE triggers cannot modify the database, they cannot activate other triggers.

  A typical use of this type of triggers is to preserve-application specific data integrity.

6. Row-level before triggers (2)

- Suppose that we want to check whether an employee's salary is greater than the salary of his or her immediate supervisor.

- Several events can trigger this rule (we assume that the salary of an employee cannot decrease):

  - inserting a new employee.
  - changing an employee's salary.
  - changing an employee's supervisor.

- Suppose that the action to take would be to call an external procedure INFORM_SUPERVISOR, which will notify the supervisor.

- The trigger can be written as follows:

6. Row-level before triggers (3)

- Trigger **R5**.

```
CREATE TRIGGER INFORM_SUPERVISOR1
BEFORE INSERT OR
        UPDATE OF SALARY, SUPERVISOR_SSN ON EMPLOYEE
FOR EACH ROW
WHEN (NEW.SALARY > (SELECT SALARY
                    FROM EMPLOYEE
                    WHERE SSN = NEW.SUPERVISOR.SSN))
INFORM_SUPERVISOR(NEW.SUPERVISOR_SSN, NEW.SSN);
```

- The **BEFORE** keyword specifies that the trigger will be fired *before* the events that trigger the rule occur.

7. Statement-level triggers (1)

- We have seen examples of triggers that have the granularity of a single row.
- However, some applications require that a trigger be fired only once per statement, after all data changes specified in the statement have been processed.
- In such cases we use statement-level triggers.

7. Statement-level triggers (2)

- Because SQL data change statements (which act as triggering events) can specify a set of tuples, one has to distinguish between whether the rule is considered *once for the whole statement* or *for each row* (tuple) affected by the statement.

- SQL allows the user to choose between these options.

- The **FOR EACH STATEMENT** clause specifies a *statement-level trigger* and can be omitted because *it is the default*.

- Note that the keywords NEW and OLD can only be used to refer to tuples with row-level triggers.

7. Statement-level triggers (3)

- Consider again the triggering event:

  1. Insert new employee tuples.

  Trigger R1 can be written as a statement-level trigger (using the syntax of SQL:1999) as follows.

  The **REFERENCING** clause can be used to name the table of NEW (OLD) tuples affected by the triggering statement in a statement level trigger.

7. Statement-level triggers (4)

- Trigger **R1S**.

```
CREATE TRIGGER TOTALSAL1S

AFTER INSERT ON EMPLOYEE
REFERENCING NEW TABLE AS  I
FOR EACH STATEMENT

WHEN EXISTS(SELECT  *
            FROM     I
            WHERE DNO IS NOT NULL )

UPDATE DEPARTMENT D
SET TOTAL_SAL = TOTAL_SAL +
                (SELECT SUM(I.SALARY)
                 FROM     I
                 WHERE  I.DNO = D.DNO)
WHERE DNO IN (SELECT DNO FROM I);
```

7. Statement-level triggers (5)

- Because multiple employee tuples may be inserted in a single insert statement, we have to check if at least one of the newly inserted employee tuples is related to a department (has a non NULL value for DNO).
- Because multiple newly inserted employees may belong to the same department, we use the SUM aggregate function.

7. Statement-level triggers (6)

- Consider also the triggering event:

  2. Change the salary of one or more existing employees.

  Trigger R2 can be written as a statement-level trigger (using the syntax of SQL:1999) as follows.

7. Statement-level triggers (7)

```
CREATE TRIGGER TOTALSAL2S

AFTER UPDATE OF SALARY ON EMPLOYEE
REFERENCING OLD TABLE AS O, NEW TABLE AS N
FOR EACH STATEMENT

WHEN EXISTS(SELECT  *
            FROM    N
            WHERE N.DNO IS NOT NULL )

UPDATE DEPARTMENT AS D
SET TOTAL_SAL = TOTAL_SAL  + (SELECT   SUM(N.SALARY)
                              FROM     N
                              WHERE    D.DNO = N.DNO)
                           –  (SELECT  SUM(O.SALARY)
                              FROM     O
                              WHERE    D.DNO = O.DNO)
WHERE DNO IN   (SELECT DNO FROM N);
```

8. Disabling triggers

- Triggers can be disabled or enabled.
- By default they are enabled when they are created but they can be disabled by using

  ALTER TRIGGER *trigger_name* **DISABLE**

- A trigger that has been disabled can be enabled again.
- A trigger can instead by dropped, which removes it permanently, by using the command

  DROP TRIGGER *trigger_name*

9. Potential applications for Active Databases

- Allowing notification of certain conditions that occur.
- Enforcing integrity constraint checking.
- Enforcing business rules.
- Updating virtual views (instead triggers).
- Maintaining derived data in a database.
- Maintaining materialized views (Data warehousing applications)
- Maintaining (remote) replicated tables